

Advanced Data Science Program
Python Module End Assessment- 1
Data Cleaning Pipeline for SaaS Dataset

Data Cleaning Pipeline for SaaS Dataset

Problem Statement

A Software-as-a-Service (SaaS) company collects customer subscription and product usage data from multiple systems, such as CRMs, billing tools, and product analytics platforms. Because the data is collected from different sources, the dataset contains several common data quality issues such as missing values, duplicate records, inconsistent date formats, and invalid numeric values.

Before performing any analytics or machine learning, the data engineering team must clean and standardize the dataset. Your task is to build a Python-based data cleaning pipeline that reads the dataset, performs cleaning operations, analyzes the data, and exports the cleaned dataset.

The program should demonstrate practical Python skills such as loops, functions, data structures, file handling, exception handling, modules, and object-oriented programming.

Students must implement the solution using Python (Google Colab or Jupyter Notebook).

Dataset:

Start your notebook with the following dictionary. Use exactly this variable name.

```
saas_data = {  
    "customer_id": [101, 102, 103, 104, 105, 106, 107],  
    "name": ["Rahul Sharma", "Meena Nair", None, "A. Kumar", "David", "Priya", "Rahul Sharma"],  
    "signup_date": ["2023-01-10", "2023/02/15", "03-01-2023", "15-04-2023", "2023-05-10",  
    "20230520", "2023-01-10"],  
    "plan": ["Pro", "Basic", "Pro", "Enterprise", "Pro", "", "Pro"],  
    "monthly_fee": ["49.99", "19.99", "forty-nine", "99.99", "49.99", "49.99", "49.99"],  
    "usage_hours": ["120", "45", "88", None, "NULL", "200", "120"]  
}  
  
print(saas_data)
```

Task 1 – Display the Dataset

Using a for loop, print each customer record in a readable format.

Example Output

ID: 101 | Name: Rahul Sharma | Signup Date: 2023-01-10 | Plan: Pro | Monthly Fee: 49.99 | Usage Hours: 120

ID: 102 | Name: Meena Nair | Signup Date: 2023/02/15 | Plan: Basic | Monthly Fee: 19.99 | Usage Hours: 45

ID: 103 | Name: None | Signup Date: 03-01-2023 | Plan: Pro | Monthly Fee: forty-nine | Usage Hours: 88

Task 2 – Add New Customer Records

- Ask the user how many customer records they want to add.
- For each new record, collect the following information:
 - Name
 - Signup Date
 - Plan
 - Monthly Fee
 - Usage Hours
- **Requirements**
 - Customer ID must auto-increment from the maximum existing ID.
 - Convert numeric inputs where necessary.
 - Append new values to the lists in `saas_data`.

Example Output:

How many customers do you want to add? 2

Enter name: Arun Kumar

Enter signup date: 2024-01-05

Enter plan: Basic

Enter monthly fee: 19.99

Enter usage hours: 60

Task 3 – Create SaaS Customer Class (OOP)

Create a class called SaaSCustomer.

Attributes:

- Customer_id
- Name
- Signup_date
- Plan
- Monthly_fee
- usage_hours

Methods:

- **validate_data()**
 - The validate_data() method should
 - Convert monthly_fee to float if possible
 - Convert usage_hours to an integer if possible
 - Replace missing name with "Unknown Customer."
 - Standardize plan names
 - Handle invalid numeric values
- **display_customer():** Display the customer name

Task 4 – Data Cleaning Pipeline

Create functions that perform the following cleaning steps.

1. Handle Missing Values

Rules:

- If the name is missing → replace it with "Unknown Customer."
- If usage_hours is NULL or missing → replace with 0
- If monthly_fee is invalid → mark it as None

2. Remove Duplicate Records

- Duplicate customers may appear in the dataset.
- Use customer_id to detect duplicates.
- Create a function: **remove_duplicates(data)**, which removes the duplicates.

3. Standardize Date Formats

- Convert signup_date to the standard format: **YYYY-MM-DD**
- Use the datetime module to parse dates.

Task 5 – Summary Statistics and Insights

Calculate and print the following:

- Total number of customers
- Average monthly fee
- Total usage hours
- Number of customers per plan

Example Output

Total Customers: 6

Average Monthly Fee: 53.79

Total Usage Hours: 573

Plan Distribution

Pro - 3

Basic - 1

Enterprise - 1

Unknown - 1

Note: The following output is calculated after cleaning and deduplication have been applied.

Task 6 – Identify Low-Usage Customers

Create a function called `find_low_usage_customers()`.

Rule: Customers with `usage_hours` less than 50 should be identified.

Example Output

Low Usage Customers

ID: 105 | Name: David | Plan: Pro | Usage Hours: 5

ID: 102 | Name: Meena Nair | Plan: Basic | Usage Hours: 10

Task 7 – Unique Plan Types

Extract unique plan types using a set and print them in alphabetical order.

Example Output

Available Plans

['Basic', 'Enterprise', 'Pro', 'Unknown']

Task 8 – Save Cleaned Data to File

- Save the cleaned dataset to a CSV file and name it **`saas_customers_cleaned.csv`**.
- Each row should contain: **`customer_id,name,signup_date,plan,monthly_fee,usage_hours`**

Example File Content

101, Rahul Sharma, 2023-01-10, Pro, 49.99, 120

102, Meena Nair, 2023-02-15, Basic, 19.99, 45

- Use try, except, else, and finally to handle file errors.
- Also, create another file named **plan_summary.txt**.

Example Content

Pro : 3

Basic: 1

Enterprise: 1

Unknown: 1

Deliverables

Students must submit a Google Drive link containing the following:

- Python Notebook (Google Colab or Jupyter Notebook) with the complete code and outputs visible.
- If using Google Colab share the notebook link with View access.
- If using Jupyter Upload the notebook to Google Drive and share the folder with View access.
- **Generated files :**
 - *Saas_customers_cleaned.csv*,
 - *plan_summary.txt*
- Place the notebook and generated files in a single Google Drive folder and share the folder link.

Skills to be Evaluated and Mapping

Assignment Section	Core Skill	Sub-Skill	Expected Proficiency
Data Setup & Load	Python basics	Lists, Dictionaries, Object creation	Beginner
Adding Records	Input & Looping	input(), append(), loop control, auto-increment	Beginner
Cleaning & Normalization	Text & Type Processing	Lowercasing, numeric parsing, date parsing	Intermediate
Deduplication & Validation	Data Integrity	Sets, dict lookups, record flags	Intermediate
Summary Statistics	Analysis	Aggregation, averages, distribution	Intermediate
OOP & Modularization	Code design	Class design, modules, import	Intermediate
File Handling & Exceptions	I/O Robustness	try/except/else/finally, CSV write	Beginner
Simple Rule-based Logic	Domain reasoning	Condition design for churn-risk	Beginner

Rubric and Mark Allocation (Total: 25 Marks)

Skill / Component	Full Marks	Criteria for Full Marks	Partial Credit Description	Partial Marks Range
Data Loading & Initial Display	3	Dataset loaded correctly, records displayed neatly, proper type conversion	Partial display or minor type conversion issues	1 – 2
Adding New Records	3	Interactive input works, auto-increment ID, correct data append	Issues in input handling, type casting, or ID logic	1 – 2
OOP Implementation	4	Class with validation, methods, and proper usage implemented	Partial class implementation or missing methods	1 – 3
Cleaning & Normalization	5	All cleaning rules correctly applied and data standardized	Partial cleaning or missed rules	1 – 4
Deduplication & Validation	3	Duplicate removal and invalid record reporting implemented	Incomplete deduplication or reporting	1 – 2
Summary Statistics	3	Accurate statistics and churn-risk logic implemented	Minor errors in calculations or logic	1 – 2

File Handling & Exception Management	3	Files created with proper exception handling	File creation correct but error handling incomplete	1 – 2
Code Quality & Documentation	1	Clean, modular code with documentation	Limited readability or missing documentation	0 – 1

Skill Taxonomy Structure

- **Data Handling:** Load and manage lists/dictionaries, iterate and add data
- **Data Cleaning:** Handle missing values, type conversion, and standardization
- **Functions & Logic:** Use functions, loops, conditions, and basic analysis
- **OOP Concepts:** Create classes, constructors, and methods
- **Data Summarization:** Count, filter, and extract unique values
- **File Handling:** Write CSV/text files and use exception handling

Evaluation Guidelines

- Run the notebook from start to finish and ensure there are no runtime errors.
- Verify the dataset dictionary is present and printed.
- Check whether new records can be added correctly.
- Inspect cleaned data for missing value handling and date normalization.
- Confirm duplicate removal logic works.
- Verify summary statistics manually for correctness.
- Ensure the files `saas_customers_cleaned.csv` and `plan_summary.txt` are generated.
- Provide marks based on the rubric and include short feedback comments.

Achievement Criteria & Passing Rules

- Minimum marks required in all sections
- Must demonstrate key skills:
 - Data Cleaning & Validation
 - Functions / Loops / OOP
- Code must be **executable and logically correct**
- The dataset must be properly cleaned and standardized
- Required files must be generated:
 - `saas_customers_cleaned.csv`
 - `plan_summary.txt`
- **Submission:** Google Drive link with notebook and files
- **Feedback:** Provide scorecard with strengths and improvements

Achievement Criteria & Passing Rules

Attempt Requirement

Each rubric component must be attempted.

Every section should receive at least partial credit.

If a section is completely missing, it will receive **0 marks**.

Minimum Competence Rule

To successfully pass the assignment, students must fully complete **at least two major components** from the following:

- Data Cleaning & Validation
- Data Analysis & Summary Statistics
- Object-Oriented Implementation (Class and Methods)
- File Handling & Report Generation

Partial Completion Policy

If a section is attempted but contains logical or minor implementation errors, **partial marks should be awarded** based on the level of correctness and effort.

Execution Requirement

The notebook must **run from start to finish without runtime errors**.

Programs that fail during execution may lose marks in the corresponding sections.

Feedback Guidelines

Evaluators should provide a **component-wise scorecard** along with brief comments describing:

- Correct implementation areas
- Logical or structural improvements required
- Coding best practices suggestions